

Maltese Snow War

Code structure, runtime flow, and P2P smoothness

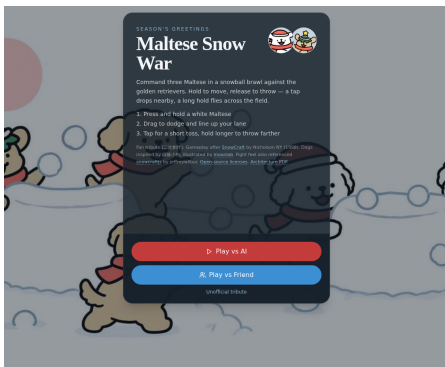
A browser remake of the 1998 Flash game SnowCraft, with moonlab's Line Puppy dogs: Maltese vs golden retrievers. Vs AI (Easy / Hard) and 1v1 WebRTC rooms.

This note maps the repo, the fixed-step game loop, and every trick used to hide network delay.

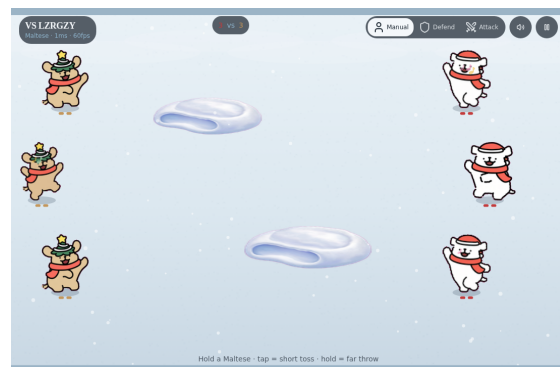
1. What this is

Maltese Snow War is a canvas snowball fight in the browser. Hold a dog, drag, release — short tap is a short throw, long hold is a long one. White Maltese (red scarves) stand on the right; brown retrievers (party hats) on the left. The host always plays the Maltese. The guest is mirrored and plays the retrievers. Two-hit bury, ellipse forts, pack-snow cooldown, and a 3-2-1 countdown before each heat. It is an unofficial fan tribute to Nicholson NY's *SnowCraft* (1998) and moonlab's Line Puppy illustrations.

Two modes: **Play vs AI** (Easy / Hard) on one machine, and **Play vs Friend** via a 6-letter room code. The host is always the white Maltese (right side). The guest's canvas is mirrored so they play the brown retrievers from the right-hand side of their screen. There is no dedicated game-simulation server — the host's browser is the authority; a small signaling server only helps the two browsers find each other.



Title card (below, not as a cover overlay): Play vs AI / Play vs Friend.



Host fight (no countdown overlay): brown retrievers left, white Maltese right. HUD: Maltese.

2. Stack

Layer	Choice	Why
UI shell	React 19 + TanStack Router + Tailwind	Title, lobby, pause, credits. Canvas sits underneath.
Game	TypeScript, Canvas 2D, fixed 1/60 s step	Deterministic-enough sim; render is a view, not the clock.
Net	WebRTC data channels + HTTP/SSE fallback	Peer-to-peer after join. TURN only if NAT blocks UDP.
Host	Vite + Nitro (Vercel preset)	SSR title page, <code>/api/rtc</code> signaling.

3. Repository map

Almost all game logic lives under `src/game/`. React is a thin chrome around one `SnowCraftGame` instance.

Path	Role
<code>src/components/game/SnowCraft.tsx</code>	React overlay: title, Easy/Hard, lobby, pause, HUD, QR.
<code>src/game/game.ts</code>	Orchestrator: screens, input, audio, net, loop.
<code>src/game/sim.ts</code>	Pure world step: throws, collisions, forts, bury.

<code>src/game/ai.ts</code>	Bot brains: idle / move / windup / dodge. Easy vs Hard.
<code>src/game/render.ts</code>	Canvas paint. Guest view is mirrored.
<code>src/game/net.ts</code>	Wire types, pack/apply pose & snapshot.
<code>src/game/versus-link.ts</code>	P2P + HTTP bus, seq, ping, clock offset.
<code>src/game/room-channel.ts</code>	SSE / poll fallback when WebRTC is down.
<code>src/lib/multiplayer/p2p.ts</code>	RTCPeerConnection mesh, ICE, data channels.
<code>src/lib/multiplayer/signaling.server.ts</code>	Room roster, ICE servers, TURN from env.
<code>src/routes/index.tsx</code>	Landing route (SSR title).
<code>src/routes/credits.tsx</code>	Licenses + tribute links.
<code>src/routes/api/rtc.ts</code>	Signaling HTTP endpoint.
<code>src/game/assets.ts / audio.ts</code>	Sprite hydrate (idle first, poses later) + SFX/BGM.

3.1 Layer cake

React chrome · SnowCraft.tsx, routes, credits

Buttons, lobby, pause — never the physics

SnowCraftGame · game.ts

Input, screens, hydrate, versus, audio unlock

Simulation · sim.ts + ai.ts

Fixed timestep. Hits, forts, bots. Authority lives here on the host.

Net · versus-link + p2p + /api/rtc

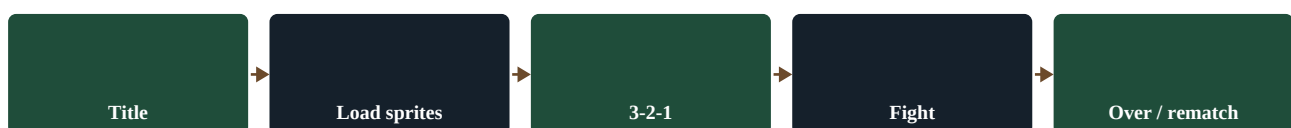
Poses, throws, hits, over. Signaling is not gameplay.

View · render.ts + sprites

Paint interpolated kids. Guest canvas is mirrored.

Data flows down: pointer → game.ts → sim (host) or local prediction (guest). Paint always reads the current GameState. The guest never decides a bury or a winner.

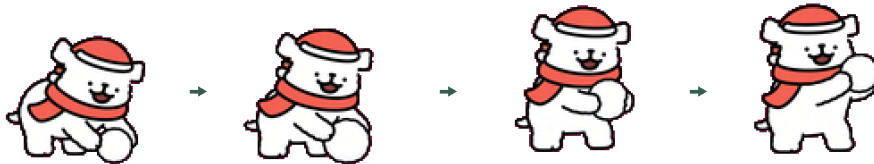
4. Screen flow





The only grey band we keep: 3-2-1 (here “PVP mode” + 3). Dogs may walk; packing is locked.

Title is React-only: no sprite preload, no AudioContext, so the first tap is live. Sprites load only after Easy/Hard, or after both PvP seats are in the room (packed handshake). Countdown lets dogs walk but not pack snow. Vs AI can return to title from pause. Vs Friend stays in the room on defeat so rematch does not need a new code.

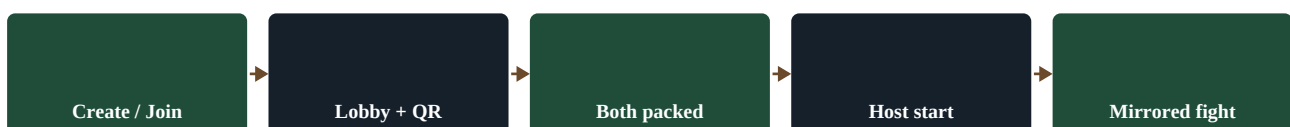


Pack-snow cooldown as a sprite strip (transparent). Four frames: crouch and wind a ball. Not in a fort.

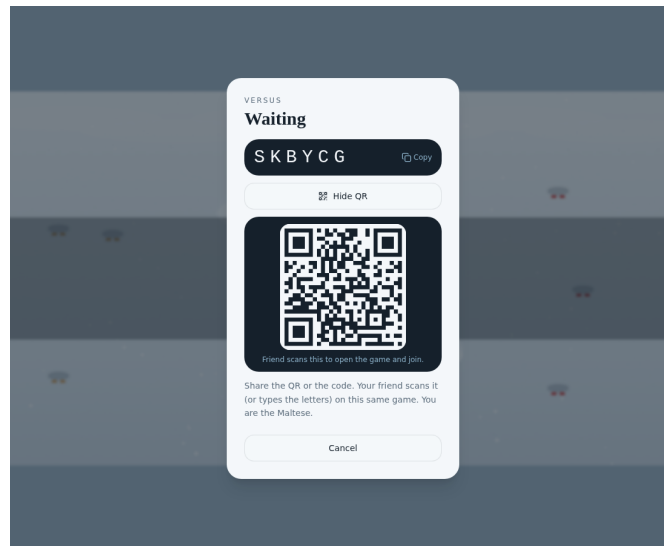


Throw cycle as a sprite strip: wind-up, release, follow-through, then the snowball. Guest predicts this locally.

4.1 Versus join



Host mints a 6-letter code. Guest types it or scans the QR. Neither side starts the heat until both have finished packing sprites. Host then sends **start**. Guest renders the yard flipped so the retrievers sit on the guest’s right.



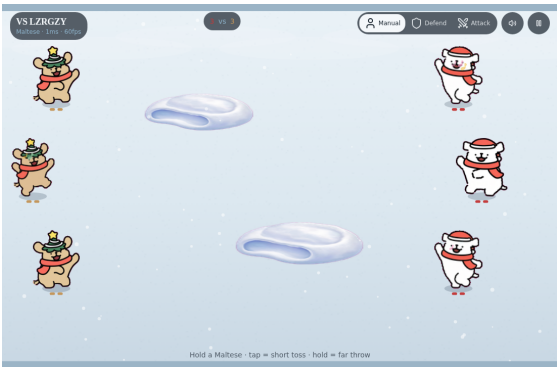
Host invite: 6-letter code plus Show QR. Friend scans the code to open this same game and join.

5. Making P2P feel smooth

This is the expensive part of the project. A 1v1 snowball fight is sparse (a few dogs, occasional throws) but unforgiving: if a hit lands a beat late, it feels broken. The host’s machine runs the real simulation. The guest’s machine predicts, interpolates, and is corrected. There is no GGPO rollback and no dedicated sim server — on purpose. Below is every technique that is actually in the code.

5.1 The unfair clock

Host-authoritative P2P is uneven by default: the host is 0 ms from the sim, the guest is one round-trip away. LAN still feels lopsided. A game server would make *both* sides guests (fairer, not always faster). For a friend-code 1v1 we instead delay the host a little and predict on the guest.



Host: Maltese (white, right). Same room code in the HUD (Maltese).



Guest: mirrored. White Maltese now on the left; retrievers on the right. HUD says Retriever.

Seat	Without help	With current netcode
Host (Maltese)	Instant move, instant throw, instant hit.	Move still instant. Throw shows a ghost ball at once; real collision waits clamp(RTT/2, 30–80 ms).
Guest (retriever)	Waits a full snapshot (~70–220 ms) to see anything.	Own move/throw are local. Opponent pose is interpolated. Hits flash locally, then confirm.

5.2 What actually goes over the wire

Early builds sent the whole yard 10–14 times a second (every kid, every ball, forts, phase). That is simple and laggy. The current split:

Message	Channel	Rate / when	Payload
pose	Unreliable (may drop)	~14–20 Hz	id, x, y, vx, vy, facing, state, hp
throw	Reliable + HTTP copy	On release	id, origin, velocity, team
hit	Reliable + HTTP copy	On damage	id, hp, heavy (bury)
snap	Unreliable keyframe	Every 1.2 s + start	Full field for recovery only
over / start / rematch / packed / bye	Reliable + unreliable + HTTP; no seq-drop	Events; over repeats 0.35 s	Winner (tiny over has no snapshot)

input	Move = pose; down/up = reliable	Guest stick + throw	x, y, hold, vx, vy, at (timestamp)
ping / pong	Unreliable	2 s	t0, t1 → RTT + clock offset

5.3 Catalogue of smoothness work

Numbered as built, not as a wish list. This is the section that answers “what did you do so PvP does not feel like a slideshow”.

1. Host input delay

Host throws are queued by $\text{clamp}(\text{RTT}/2, 30\text{--}80)$ ms before they enter the authoritative sim. Guest input already paid the network RTT, so both throws land in a similar window. Movement stays immediate so the dog still follows the finger.

2. Host ghost ball (local prediction)

On release the host spawns a ghost snowball: it flies and paints, but does not collide. When the delay elapses the ghost is killed and a real ball is committed + broadcast. The host sees an instant throw; fairness stays in the delayed hit test.

3. Guest local move & throw

Guest dogs follow the pointer with no round-trip. Guest throws spawn local balls immediately and also send input.at to the host. Pose packets do not overwrite the grabbed dog.

4. Guest predicted hits

If a local ball overlaps an enemy, the guest flashes hurt + SFX at once, without changing HP. A matching hit packet confirms (HP/bury). If nothing arrives in ~140 ms, the flash is rolled back. Juice is instant; score is still host-declared.

5. Pose velocity / dead reckoning

Each pose carries vx, vy (pixels per second from the last sample). The guest eases toward pos + v·lead, with lead capped ~50 ms — enough to hide 14 Hz gaps, not enough to cancel the host delay (which would restore host advantage).

6. Clock offset

Ping carries t0; pong echoes t0 and t1. Each side estimates $\text{peerTime} - \text{localTime}$, smoothed. Guest uses $\text{hostNow} \approx \text{now} + \text{offset}$ when applying poses. Guest inputs carry at; the host maps them into its delay buffer instead of applying wall-clock-now.

7. Keyframes do not clobber balls

The 1.2 s snapshot repairs kids and HP. Local and ghost balls are kept unless a host ball from the same thrower is already nearby. Predicted throws no longer pop out of existence.

8. Two data channels + HTTP belt

Unreliable channel: poses, pings (drop stale). Ordered reliable: throws, hits, over. over / start / rematch / packed / bye also copy onto the unreliable channel and HTTP/SSE, and they ignore the sequence filter so a late snapshot cannot eat the winner.

9. Outcome is not a snapshot

A fat WireState over packet was easy to drop. Now a tiny { winner } is sent as well, resent every 0.35 s, and poses keep advertising phase won/lost after the heat. Guest can infer the result three ways. Rematch start is recovered from a fight pose if both players already voted yes.

10. Packed handshake

The match does not start until both browsers have the pose sprites. Prevents one side fighting blobs while the other has dogs, which felt like desync.

11. ICE / TURN / relay HUD

STUN first; ExpressTURN from server env (never committed). ICE restart if stuck; fall back to SSE poll. HUD shows direct vs relay, RTT, fps so we can tell lag from art.

12. Guest-local presentation

Idle fidget, walk cycle, aim line, particles, SFX, and 3-2-1 ticks run locally. They are not networked. Traffic stays tiny; the yard still looks alive.

13. Ally bots on the guest

Defend / Attack for unselected Maltese (or retrievers on guest) step locally; poses and throws are sent up. The guest does not wait for the host to wiggle idle dogs.

14. What we did not do

No GGPO rollback (sim is not deterministic: Math.random, wall clocks). No dedicated game server (1v1 friend rooms; signaling is enough). No binary protobuf yet — JSON poses are already small. Those remain valid later if this becomes ranked matchmaking.

5.4 Throw path (both seats)



Guest: local ball + predicted flash, then wait for hit. Host: ghost ball, then delayed real ball, then hit + pose. Either winner packet or a won/lost pose ends the heat on both screens.



Same throw strip on the wire path: local ball first, host collision later.

5.5 Why not a sim server

A server would equalize latency (both sides ~RTT to Tokyo/Singapore) and help anti-cheat. It would not make LAN faster, and it would add 24/7 ops. For a 6-dog yard, pose+event+prediction is the right cost curve. Revisit a server if we add public matchmaking or need to stop a host from lying about HP.

6. Vs AI (for contrast)

Easy: original pacing, nearest-target bots, forts are cover only, Maltese revive next heat. Hard: bot move $\times 3$, smarter dodge, forts have 10 HP, no Maltese revive, Maltese move $\times 2$ and both teams' snowballs $\times 2$. Same sim as PvP; no net. Difficulty is chosen on a gate after Play vs AI, which is also when sprites load (progress bar).

7. Loading strategy

SSR paints a Loading spinner until React hydrates, then the title card. Play vs AI / Friend are real buttons only after hydrate. hydrateCore (idle dogs + fort) and hydrateRest (throw/hurt/walk/...) run after a mode is chosen. Unused placeholder characters were removed so the landing wait is not packing dead art.

8. Trust and secrets

TURN username/credential live in server env / local .env, never in git. The public repo only has empty TURN_USERNAME / TURN_CREDENTIAL in .env.example. P2P gameplay is host-honest: a cheating host can still lie. Fine for friends; not for ranked.

9. Credits (short)

Game: Nicholson NY, SnowCraft, 1998. Characters: moonlab / Line Puppy (fan illustration). Fight feel referenced jeffreywilbur/snowcraftjs. Libraries (MIT / Apache-2.0 / ISC / OFL) are listed in-app at /credits.

End of briefing. Source of truth is the beta branch of the GitHub repo; this PDF describes the P2P pose/prediction stack as of August 2026.